

Unifying Uniform and Binary-coding Quantization for Accurate Compression of Large Language Models

Anonymous ACL submission

Abstract

How can we quantize large language models while preserving accuracy? Quantization is essential for deploying large language models (LLMs) efficiently. Binary-coding quantization (BCQ) and uniform quantization (UQ) are promising quantization schemes that have strong expressiveness and optimizability, respectively. However, neither scheme leverages both advantages. In this paper, we propose **UniQuan_F** (**Unified Quantization with Flexible Mapping**), an accurate quantization method for LLMs. UniQuan_F harnesses both strong expressiveness and optimizability by unifying the flexible mapping technique in UQ and BCQ’s non-uniform quantization levels. We propose unified initialization, and local and periodic mapping techniques to optimize the parameters in UniQuan_F precisely. After optimization, our unification theorem removes computational and memory overhead, allowing us to utilize the superior accuracy of UniQuan_F without extra deployment costs induced by the unification. Experimental results demonstrate that UniQuan_F outperforms existing UQ and BCQ methods, achieving up to 4.60% higher accuracy on GSM8K benchmark.

1 Introduction

How can we compress large language models without compromising accuracy? Reducing the size of large language models (LLMs) (Brown et al., 2020; Touvron et al., 2023; Dubey et al., 2024) is crucial for deploying them in real-world applications since they require expensive computational and memory costs. Quantization algorithms (Xu et al., 2018; Kwon et al., 2022; Dettmers et al., 2022; Xiao et al., 2023; Lin et al., 2024) efficiently compress LLMs via bit-width reduction of weights by encoding them with a small set of values, namely, quantization levels. Each quantization scheme has its own quantization parameters that determine the quantization levels; accurately optimizing these pa-

rameters is important because it guarantees that the quantization levels are well aligned with the model’s weight distribution.

Uniform quantization (UQ) (Lee et al., 2023; Lin et al., 2024; Shao et al., 2024) and binary-coding quantization (BCQ) (Xu et al., 2018; Kwon et al., 2022) are promising quantization schemes that ensure fast inference of quantized LLMs. While UQ divides its quantization levels uniformly, BCQ results in non-uniform quantization levels through the addition and subtraction of values assigned per each bit. Research on quantizing LLMs with UQ is actively ongoing, but no study has yet explored BCQ on LLMs as fast acceleration kernels (Park et al., 2024; You et al., 2024) that support BCQ scheme have been released recently.

We compare the expressiveness and optimizability of quantization schemes for LLMs in Figure 1. UQ demonstrates strong optimizability, efficiently reducing quantization errors, by leveraging various advanced methods such as FlexRound (Lee et al., 2023) and OmniQuant (Shao et al., 2024). However, its expressiveness is constrained, failing to adapt its quantization levels to the weight distribution due to their uniform spacing. Conversely, BCQ offers strong expressiveness through non-uniform quantization levels, but its optimizability is limited due to the absence of precise quantization techniques. ALTERNATING (Xu et al., 2018) is the only BCQ method applicable to LLMs, but its major drawback is that it does not consider the input distribution. In summary, neither scheme fully achieves both strong expressiveness and optimizability.

In this paper, we propose **UniQuan_F** (**Unified Quantization with Flexible Mapping**), an accurate quantization method for LLMs. We find that the strong optimizability of UQ originates from its transformation process, and the strong expressiveness of BCQ originates from its generalized mapping process that maps weights to non-uniform quantization levels (see Section 3.1). Building

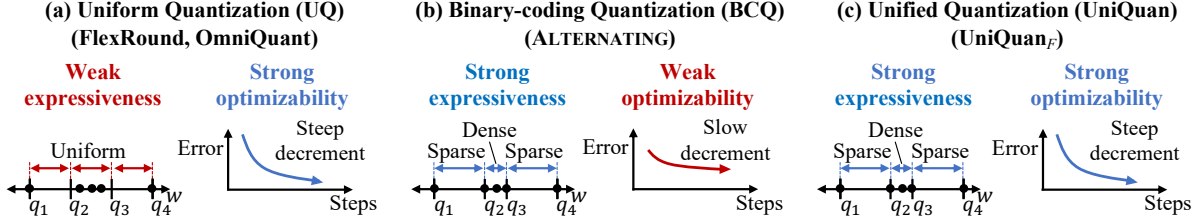


Figure 1: A comparison of (a) uniform, (b) binary-coding, and (c) unified quantization schemes. UQ has strong optimizability swiftly reducing errors during optimization, and BCQ has strong expressiveness adapting its non-linear quantization levels (q , bar) according to the distribution of weights (w , circle). UniQuan combines the advantages of both schemes by unifying UQ and BCQ.

on this observation, we define **UniQuan** (**Unified Quantization**) which unifies UQ and BCQ schemes by integrating UQ’s transformation process into BCQ’s mapping process, thereby harnessing both strong optimizability and expressiveness as shown in Figure 1(c). In UniQuan_F, we unify FlexRound and ALTERNATING, the best-performing UQ and BCQ methods, respectively, following the quantization process of UniQuan. We improve the accuracy of UniQuan_F with two main ideas: 1) unified initialization for joint initialization of quantization parameters from FlexRound and ALTERNATING, and 2) local and periodic mapping to accelerate the slow mapping process of BCQ. We further remove the extra deployment cost caused from the unification by integrating the two-step inference process into a single step with unification theorem (see Theorem 1). As a result, UniQuan_F exhibits the best performance without any additional memory and computational costs at deployment.

Our main contributions are as follows:

- **Algorithm.** We propose UniQuan_F, an accurate quantization method for LLMs. UniQuan_F unifies the best-performing UQ and BCQ methods to leverage both of their advantages.
- **Analysis.** We analyze that UniQuan_F exhibits both strong expressiveness and optimizability.
- **Experiments.** We show that UniQuan_F outperforms UQ and BCQ methods on various benchmarks, showing up to 4.60% higher accuracy. The source code is publically available at <https://github.com/anony-authors/UniQuanF>.

2 Preliminary

We introduce the LLM quantization problem and describe preliminaries. We describe the frequently used terminologies in Appendix A.

2.1 LLM Quantization Problem

We have a pre-trained LLM F , a desired bit-width k , and a sample dataset $\mathbb{D}$. Our objective is to

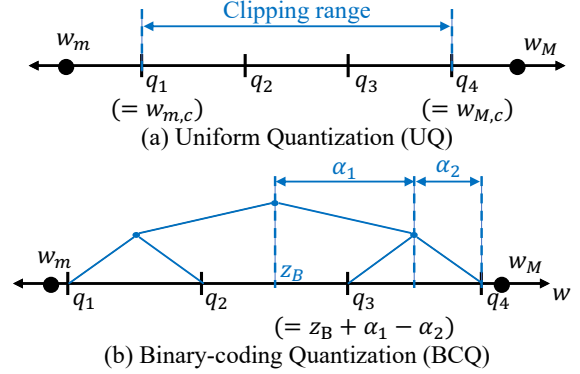


Figure 2: Illustrations of quantization levels (q) assigned for a weight group w under (a) UQ and (b) BCQ schemes, where w_m and w_M are the minimum and the maximum weights in w , respectively. UQ has evenly-spaced quantization levels within a clipping range while BCQ has non-uniform quantization levels determined by its scale factors α and a shifting factor z_B .

find an accurate k -bit quantized model $\hat{F}$. The target model is a Transformer-based (Vaswani et al., 2017) LLM (Jiang et al., 2023; Dubey et al., 2024) with N blocks, where quantization is applied to the weight matrices in each block. We divide each weight matrix into multiple weight groups that share quantization parameters. Given a group $w \in \mathbb{R}^g$ of g weight values and a desired bit-width k , a quantizer Q quantizes w into $\hat{w} = Q(w, k; \Theta)$ where Θ is a set of quantization parameters. In this paper, we focus on two quantization schemes: uniform quantization (UQ) and binary-coding quantization (BCQ) which are supported by the state-of-the-art inference kernel (Park et al., 2024). We directly compare their accuracy since they require the same memory and computational costs.

2.2 Uniform Quantization (UQ)

Uniform quantization (UQ) (Lee et al., 2023; Shao et al., 2024) is a quantization scheme that has uniformly spaced quantization levels. We explain UQ using Round-to-nearest (RTN) (Gupta et al., 2015), a representative method of UQ. Figure 2(a) illus-

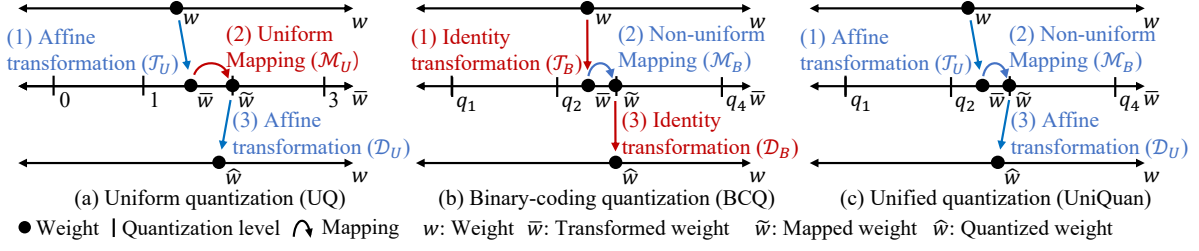


Figure 3: Quantization processes for a weight w in (a) UQ, (b) BCQ, and (c) UniQuan schemes. Blue-colored processes are parameterized functions which are the source of UQ’s strong optimizability and BCQ’s powerful expressiveness, respectively. UniQuan takes the advantages of both UQ and BCQ schemes by combining the parameterized functions in both schemes. See Section 3.1 for details.

trates the quantization levels in UQ that are evenly distributed in a clipping range $[w_{m,c}, w_{M,c}]$, where $w_{m,c}$ and $w_{M,c}$ are the minimum and maximum values in the clipping range, respectively. RTN begins with a grid search to find the proper $w_{m,c}$ and $w_{M,c}$ to precisely approximate the original weights. The quantization parameters $\Theta_R = \{\Delta, z_U\}$ of RTN are derived based on its clipping range; scale factor $\Delta = (w_{M,c} - w_{m,c}) / (2^k - 1)$ and zero-point $z_U = \lfloor -w_{m,c} / \Delta \rfloor$, where $\lfloor \cdot \rfloor$ is the rounding function. Then, RTN quantizer Q_R quantizes $w \in \mathbb{R}^g$ into $\hat{w} = Q_R(w, k; \Theta_R)$ as follows:

$$\begin{aligned} \tilde{w} &= \text{Clip}(\lfloor w / \Delta + z_U \mathbf{1}_g \rfloor, 0, 2^k - 1), \\ \hat{w} &= \Delta(\tilde{w} - z_U \mathbf{1}_g), \end{aligned}$$

where $\text{Clip}(\cdot, m, M)$ is an element-wise clipping function with a min-max range $[m, M]$, and $\tilde{w}$ is a vector of low-bit integers assigned for each weight. $\mathbf{1}_g$ is a vector of size g filled with ones.

2.3 Binary-coding Quantization (BCQ)

Binary-coding quantization (BCQ) is a non-uniform quantization scheme with adaptive quantization levels. Its quantization parameters $\Theta_B = \{\alpha, z_B\}$ consists of a vector $\alpha \in \mathbb{R}^k$ of scale factors and a shifting factor $z_B \in \mathbb{R}$. Figure 2(b) illustrates the quantization levels in BCQ that are determined through the summation and subtraction of scale factors α after shifting with z_B . In this example, BCQ quantizer defines the set $\{z_B + \alpha_1 + \alpha_2, z_B + \alpha_1 - \alpha_2, z_B - \alpha_1 + \alpha_2, z_B - \alpha_1 - \alpha_2\}$ of four quantization levels forming a binary tree centered at z_B with widths α_1 and α_2 .

We assign a binary code $c \in \{-1, +1\}^k$ for each weight w , indicating whether the weight selects the positive or negative value for each scale factor in α . Then, BCQ quantizer Q_B quantizes a weight group $w \in \mathbb{R}^g$ into $\hat{w} = Q_B(w; \Theta_B)$ as follows:

$$\hat{w} = C\alpha + z_B \mathbf{1}_g,$$

$$\text{where } C = \arg \min_{C'} \|w - (C'\alpha + z_B \mathbf{1}_g)\|_2^2.$$

$C \in \{-1, +1\}^{g \times k}$ is the binary code matrix for w where its i th row contains the binary code for the i th weight in w . Existing works (Xu et al., 2018) first set z_B as 0, then search the scale factors α and binary codes C through an alternating update process (see Algorithm 2 in Xu et al. (2018)).

The main advantage of BCQ is its strong expressiveness; any UQ is representable in the form of BCQ (see Appendix C in Park et al. (2024)). However, BCQ methods are far less accurate in quantizing LLMs than UQ methods due to the lack of accurate BCQ-based quantization methods. In this paper, we propose UniQuan_F which transfers UQ’s accurate quantization methods to BCQ to fully exploit BCQ’s strong expressiveness.

3 Proposed Method

We propose **UniQuan_F** (**Unified Quantization with Flexible Mapping**), an accurate quantization method for LLMs. We first propose **UniQuan** (**Unified Quantization**), a framework unifying the quantization processes of UQ and BCQ schemes to exploit their strong expressiveness and optimizability. Then, we present UniQuan_F-0, a naïve method that unifies FlexRound and ALTERNATING following UniQuan, but with several limitations. Finally, we propose UniQuan_F which achieves high accuracy without requiring extra costs at deployment, by addressing the limitations in UniQuan_F-0.

3.1 Unification of UQ and BCQ

We define a general representation of the quantization processes in UQ and BCQ schemes to clarify the source of their superior capabilities. Given a quantizer Q with quantization parameters Θ , the general representation is defined as follows:

$$\hat{w} = Q(w; \Theta) = \mathcal{D}(\mathcal{M}(\mathcal{T}(w; \Theta); \Theta); \Theta),$$

where $\mathcal{T}$ is a transformation process which transforms weights w to transformed weights $\bar{w}$, $\mathcal{M}$ is a mapping process which maps transformed weights

Table 1: Comparison of quantization methods. UniQuan_F incorporates the advantages of FlexRound and ALTERNATING by exploiting $\mathcal{T}_F$, $\mathcal{D}_R$, and $\mathcal{M}_B^*$, overcoming the challenges in UniQuan_{F-0}. $\circ$ is a function composition operator. See Sections 3.2 for details.

Method	Optimization		Deployment	
	Initialization	Quantization	Parameters	Inference
FlexRound	Grid search	$\mathcal{D}_R \circ \mathcal{M}_U \circ \mathcal{T}_F$	$\tilde{\mathbf{w}}, \Theta_R$	$\mathcal{D}_R(\tilde{\mathbf{w}}; \Theta_R)$
ALTERNATING	Alternating update	$\mathcal{M}_B$	$\mathbf{C}, \Theta_B$	$\mathcal{R}_B(\mathbf{C}; \Theta_B)$
UniQuan _{F-0}	-	$\mathcal{D}_R \circ \mathcal{M}_B \circ \mathcal{T}_F$	$\mathbf{C}, \Theta_B, \Theta_R$	$\mathcal{D}_R(\mathcal{R}_B(\mathbf{C}; \Theta_B); \Theta_R)$
UniQuan_F (Proposed)	Unified initialization	$\mathcal{D}_R \circ \mathcal{M}_B^* \circ \mathcal{T}_F$	$\mathbf{C}, \Theta_B^*$	$\mathcal{R}_B(\mathbf{C}; \Theta_B^*)$

$\tilde{\mathbf{w}}$ to the corresponding quantization levels, and $\mathcal{D}$ is a detransformation process which reverts mapped weights $\tilde{\mathbf{w}}$ to the original weight space, yielding quantized weights $\hat{\mathbf{w}}$. We illustrate the quantization processes in diverse quantization schemes in Figure 3 following this general representation.

Optimizability of UQ. As shown in Figure 3(a), we formulate the quantizer Q_U of UQ as follows: $Q_U(\mathbf{w}, k; \Theta_U) = \mathcal{D}_U(\mathcal{M}_U(\mathcal{T}_U(\mathbf{w}; \Theta_U), k); \Theta_U)$, where $\mathcal{M}_U(\mathbf{w}, k) = \text{Clip}(\lfloor \mathbf{w} \rfloor, 0, 2^k - 1)$ is a uniform mapping function that maps transformed weights $\tilde{\mathbf{w}}$ to uniform quantization levels. $\mathcal{T}_U$ and $\mathcal{D}_U$ are affine transformation functions with quantization parameters Θ_U . There are two main methods belonging to UQ: RTN and FlexRound. In RTN, the quantizer Q_R is formulated as follows:

$Q_R(\mathbf{w}, k; \Theta_R) = \mathcal{D}_R(\mathcal{M}_U(\mathcal{T}_R(\mathbf{w}; \Theta_R), k); \Theta_R)$, where $\Theta_R = \{\Delta, z_U\}$, $\mathcal{T}_R(\mathbf{w}; \Theta_R) = \mathbf{w}/\Delta + z_U \mathbf{1}_g$ and $\mathcal{D}_R(\mathbf{w}; \Theta_R) = \Delta(\mathbf{w} - z_U \mathbf{1}_g)$. In FlexRound, the main idea is to allow each weight to explore diverse quantization levels to find the optimal one. This is achieved by replacing the transformation function $\mathcal{T}_R$ with $\mathcal{T}_F$ defined as follows:

$$\mathcal{T}_F(\mathbf{w}; \Theta_F) = \mathbf{w} \oslash (\Delta \cdot \mathbf{s} \cdot \mathbf{s}_r) + z_U \mathbf{1}_g, \quad (1)$$

where $\Theta_F = \{\Delta, z_U, \mathbf{s}, \mathbf{s}_r\}$ is a set of quantization parameters of FlexRound, and $\oslash$ denotes an element-wise division. $\mathbf{s} \in \mathbb{R}^g$ and $\mathbf{s}_r \in \mathbb{R}$ are element-wise and row-wise scale factors to promote the exploration of weights, respectively.

In summary, each quantization method has its own Θ_U , $\mathcal{T}_U$, and $\mathcal{D}_U$, and they are the origin of strong optimizability. We formulate the quantizers of various UQ methods in Appendix E.6.

Expressiveness of BCQ. As shown in Figure 3(b), we formulate the quantizer Q_B of BCQ as follows:

$$Q_B(\mathbf{w}; \Theta_B) = \mathcal{D}_B(\mathcal{M}_B(\mathcal{T}_B(\mathbf{w}); \Theta_B)),$$

where $\mathcal{T}_B(\mathbf{w}) = \mathcal{D}_B(\mathbf{w}) = \mathbf{w}$ are identity functions. $\mathcal{M}_B(\mathbf{w}; \Theta_B) = Q_B(\mathbf{w}; \Theta_B)$ is a non-uniform mapping function that maps transformed

weights $\tilde{\mathbf{w}}$ to the nearest non-uniform quantization levels. $\mathcal{M}_B$ enables the mapping between weights and non-uniform quantization levels, enabling strong expressiveness. ALTERNATING (Xu et al., 2018) is the only quantization method in BCQ scheme applicable for LLMs with the same quantizer Q_B defined above.

UniQuan (Unified Quantization). We propose UniQuan, a framework unifying the quantization processes of UQ and BCQ schemes. UniQuan formulates the unified quantizer Q_I with quantization parameters $\Theta_I = \{\Theta_B, \Theta_U\}$ as follows (see Figure 3(c)):

$$Q_I(\mathbf{w}; \Theta_I) = \mathcal{D}_U(\mathcal{M}_B(\mathcal{T}_U(\mathbf{w}; \Theta_U); \Theta_B); \Theta_U).$$

Q_I has strong expressiveness by mapping weights to the non-uniform quantization levels using $\mathcal{M}_B$. Also, Q_I has strong optimizability by incorporating useful methods in UQ by replacing Θ_U , $\mathcal{T}_U$, and $\mathcal{D}_U$ with those of the method to incorporate.

3.2 UniQuan_{F-0}

We introduce a basic method UniQuan_{F-0} which naively unifies the best-performing UQ and BCQ methods, FlexRound (Lee et al., 2023) and ALTERNATING (Xu et al., 2018), respectively. The quantizer Q_{I_F} of UniQuan_{F-0} is defined as follows:

$$\begin{aligned} Q_{I_F}(\mathbf{w}; \Theta_{I_F}) \\ = \mathcal{D}_R(\mathcal{M}_B(\mathcal{T}_F(\mathbf{w}; \Theta_F); \Theta_B); \Theta_R). \end{aligned} \quad (2)$$

Note that FlexRound begins its optimization process by initializing its quantization parameters with a grid search process to find a proper clipping range. FlexRound enhances its optimizability by utilizing $\mathcal{T}_F$ in Equation 1 and optimizes its quantization parameters Θ_F using stochastic gradient descent (SGD). After optimization, FlexRound stores low-bit weights $\tilde{\mathbf{w}} = \mathcal{M}_U(\mathcal{T}_F(\mathbf{w}; \Theta_F), k)$ and quantization parameters $\Theta_R = \{\Delta, z_U\}$, and approximates the original weights $\mathbf{w}$ as $\hat{\mathbf{w}} = \mathcal{D}_R(\tilde{\mathbf{w}}; \Theta_R)$ in the detransformation step; $\mathbf{s}$ and $\mathbf{s}_r$ in Θ_F are

used only for transformation. On the other hand, ALTERNATING initializes its quantization parameters in $\Theta_B = \{\alpha, z_B\}$ with alternating update (Xu et al., 2018). Then, it stores the binary code matrix C and quantization parameters Θ_B . It approximates weights w as $\hat{w} = \mathcal{R}_B(C; \Theta_B) = C\alpha + z_B \mathbf{1}_g$ for inference.

UniQuan_F-0 inherits the strong optimizability from FlexRound’s $\mathcal{T}_F$ and $\mathcal{D}_R$, and the strong expressiveness from ALTERNATING’s $\mathcal{M}_B$. However, UniQuan_F-0 has three points of improvements as follows:

C1. Joint initialization. FlexRound and ALTERNATING are based on different initialization methods which seem incompatible. How can we jointly initialize the quantization parameters considering their dependency?

C2. Slow mapping process. The quantization levels of UniQuan_F-0 are changed during optimization process where its quantization parameters are updated. Therefore, it is crucial to update the mappings between weights and quantization levels during optimization using $\mathcal{M}_B$. However, existing method (Xu et al., 2018) exploits a binary search tree (BST) to implement $\mathcal{M}_B$ which finds the mappings through a GPU-unfriendly sequential process, and it requires intractable time for updating mappings during optimization. How can we expedite the mapping process?

C3. Expensive deployment cost. UniQuan_F-0 suffers from memory and computational overheads at deployment, since UniQuan_F-0 require executing both inference procedures $\mathcal{D}_R$ and $\mathcal{R}_B$ (see Table 1). How can we decrease the inference cost?

3.3 UniQuan_F

We propose UniQuan_F (Unified Quantization with Flexible Mapping), which improves UniQuan_F-0 with the following main ideas.

I1. Unified initialization. We unify the initialization algorithms in FlexRound and ALTERNATING to jointly initialize the quantization parameters, considering their dependency.

I2. Local and periodic mapping. We efficiently update mappings by exploiting the locality and the temporal sparsity of changes in mappings.

I3. Unification theorem. We unify the inference process of UniQuan_F-0 into a single BCQ’s inference process, removing additional memory and computational costs at deployment.

Algorithm 1 UniQuan_F for a block f

Input: A Transformer block f , f ’s input X , a set Φ of weight groups in f , the input $\hat{X}$ of a quantized f obtained by preceding quantized blocks, number E of epochs, and number $|\mathbb{D}|$ of data points

Output: Optimized quantization parameters Θ_B^* and a binary-code matrix C for each group

- 1: Initialize Θ_{I_F} for each weight group
 $\triangleright$ (I1) Unified Initialization
 - 2: **for** epoch e in 1, 2, ..., E **do**
 - 3: **for** s in 1, 2, ..., $|\mathbb{D}|$ **do**
 - 4: Quantize Φ into $\hat{\Phi}$ using Equation 2
 $\triangleright$ (I2) Local and periodic mapping
 - 5: $\mathcal{L} \leftarrow \|f(X_s; \Phi) - f(\hat{X}_s; \hat{\Phi})\|_F^2$
 - 6: Perform backpropagation using $\mathcal{L}$
 - 7: Update Θ_{I_F} for each group
 - 8: **end for**
 - 9: **end for**
 - 10: Convert each Θ_{I_F} into $\Theta_B^* = \{\alpha^*, z_B^*\}$
 $\triangleright$ (I3) Unification Theorem
 - 11: Find C using Equation 4 for each group
-

UniQuan_F quantizes an LLM through a block-wise optimization strategy, quantizing sequentially from the bottom-most Transformer block upwards. Algorithm 1 details the process of quantizing a Transformer block f with a set Φ of weight groups, to obtain a binary-code C and optimized quantization parameters Θ_B^* for each group. We provide inputs X and $\hat{X}$ of unquantized and quantized blocks, respectively. We iterate the optimization process for E epochs using $|\mathbb{D}|$ data points.

First, for each weight group, we initialize the quantization parameters $\Theta_{I_F} = \Theta_B \cup \Theta_F$ using the unified initialization technique (line 1) which we describe later in this subsection. Subsequently, we iteratively optimize the quantization parameters to minimize the distance between the outputs of the quantized and unquantized blocks (lines 2-9). In each optimization step, we quantize the weight groups in Φ using UniQuan_F’s quantizer $Q_{I_F}^*$ with its quantization parameters Θ_{I_F} as follows:

$$Q_{I_F}^*(w; \Theta_{I_F}) = \mathcal{D}_R(\mathcal{M}_B^*(\mathcal{T}_F(w; \Theta_F); \Theta_B); \Theta_R), \quad (3)$$

where $\mathcal{M}_B^*$ represents the local and periodic mapping technique, which we describe later in this subsection, to accelerate mappings to non-uniform quantization levels. $\mathcal{T}_F$ is the transformation function of FlexRound in Equation 1. We quantize each weight group in parallel, and the quantized

Table 2: The percentage of index changes per optimization step and throughout all 2,560 steps. Minimal changes in each step add up to substantial shifts overall.

Step size	Index change			
	0	1	2	>2
Single step	99.97%	0.01%	0.00%	0.01%
All steps (2,560)	89.30%	8.37%	1.36%	0.97%

weight groups are stored in $\widehat{\Phi}$ (line 4). We measure the reconstruction loss $\mathcal{L}$ which represents the distance between the outputs of the original and quantized blocks, and then optimize the quantization parameters Θ_{I_F} via stochastic gradient descent (lines 5-7). After the iterative optimization process, we have the optimized quantization parameters $\Theta_{I_F} = \Theta_B \cup \Theta_F$ which generate the outputs closely resembling those of the original block. We leverage the unification theorem (Theorem 1) to merge the optimized quantization parameters into a single set of BCQ’s quantization parameters $\Theta_B^* = \{\alpha^*, z_B^*\}$ (line 10). Finally, we obtain a binary-code matrix C using Θ_B^* for each weight group as follows:

$$C = \arg \min_{C'} \|\mathbf{w} - (C' \alpha^* + z_B^* \mathbf{1}_g)\|_2^2. \quad (4)$$

Then, the quantization process for a block is completed (line 11). We obtain an entirely quantized model by sequentially applying Algorithm 1 from the bottom to the top blocks.

Unified Initialization. How can we jointly initialize the quantization parameters of FlexRound and ALTERNATING? FlexRound initializes Δ and z_U through a grid search process, and ALTERNATING utilizes an alternating update process to find α , fixing z_B as 0. We propose a unified initialization process which initializes the quantization parameters $\Theta_{I_F} = \{\Delta, z_U, \mathbf{s}, s_r, \alpha, z_B\}$ by unifying both techniques, considering their dependency.

In unified initialization, we perform alternating updates to find α and z_B in each iteration of a grid search process for Δ and z_U . As a result, we find α and z_B according to the Δ and z_U during the grid search process. As in FlexRound, we initialize $\mathbf{s}$ and s_r as $\mathbf{1}_g$ and 1, respectively. We detail the unified initialization algorithm in Appendix E.1.

Local and Periodic Mapping. How can we swiftly update the mappings between weights and quantization levels during optimization? Table 2 shows that while over 8% of weights change indices of the mapped quantization levels during the full optimization, 99.97% remain stable at each

step, and most changes involve adjacent levels. Hence, we propose a local and periodic mapping which evaluates only neighboring levels and remaps periodically. We set a hyperparameter p as a remapping period, and in every p steps, we find the closest quantization level among the previous level and its two neighbors. We detail the process of local and periodic mapping in Appendix E.4.

Unification Theorem. How can we eliminate the memory and computational overheads at deployment induced by the unification? We propose the unification theorem that integrates the two-step inference process of UniQuan_F-0 into a single BCQ inference process as follows:

Theorem 1 (Unification Theorem). *Given a reconstruction function $\mathcal{R}_B(C; \Theta_B)$ and a detransformation function $\mathcal{D}_R(\tilde{\mathbf{w}}; \Theta_R)$ where $\tilde{\mathbf{w}} = \mathcal{R}_B(C; \Theta_B)$, there is a set Θ_B^* of unified quantization parameters such that $\mathcal{R}_B(C; \Theta_B^*) = \mathcal{D}_R(\mathcal{R}_B(C; \Theta_B); \Theta_R)$ for any C , Θ_B , and Θ_R .*

Proof. See Appendix E.5. $\square$

As a result, UniQuan_F exploits the UQ’s optimizability without any extra costs at deployment.

4 Experiments

We perform experiments to answer the following questions. Additional analyses on UniQuan_F are discussed in Appendix F.

Q1. General knowledge evaluation. How accurately does UniQuan_F quantize LLMs on general knowledge benchmarks?

Q2. Task-specific knowledge evaluation. How accurately does UniQuan_F quantize LLMs on task-specific knowledge benchmarks?

Q3. Ablation study. Do all ideas in UniQuan_F improve the accuracy of quantized LLMs?

Q4. Case study. Does UniQuan_F effectively utilize BCQ’s expressiveness and UQ’s optimizability as we intended?

4.1 Experimental Setup

We briefly introduce the experimental setup. Refer to Appendix C for further details.

Setup. We evaluate the quantized performance of Llama-3 8B, Llama-3 70B (Dubey et al., 2024), and Mistral 7B (Jiang et al., 2023) models on MMLU (Hendrycks et al., 2021) and WikiText2 (Merity et al., 2017) benchmarks. For task-specific experiment, we quantize Llama-3 8B Instruct (Dubey et al., 2024) model and evaluate on

Table 3: Comparison of the average accuracy of 0-shot and 5-shot on MMLU, and the perplexity on WikiText2 (Wiki) benchmarks. Higher accuracies and lower perplexities indicate better performance. Bold and underlined texts indicate the best and second-best performance, respectively. UniQuan_F shows the best performance in most cases.

# Bits	Scheme [†]	Method	Mistral 7B		Llama-3 8B		Llama-3 70B	
			MMLU	Wiki	MMLU	Wiki	MMLU	Wiki
			Avg. (↑)	(↓)	Avg. (↑)	(↓)	Avg. (↑)	(↓)
Full precision			61.40	5.25	62.77	6.14	77.52	2.86
4	UQ	RTN	55.20	6.04	54.67	7.80	67.74	3.85
		OmniQuant	56.28	5.67	56.02	<u>7.37</u>	76.62	3.34
		FlexRound	<u>58.60</u>	<u>5.52</u>	<u>58.59</u>	8.22	77.24	<u>3.31</u>
	BCQ	ALTERNATING	<u>24.65</u>	9.8e4	<u>22.95</u>	1.3e5	<u>44.07</u>	<u>11.06</u>
		UniQuan_F	59.22	5.51	61.43	7.01	<u>76.97</u>	3.19
3	UQ	RTN	28.59	36.07	24.35	120.50	40.61	24.42
		OmniQuant	32.82	9.24	26.34	42.06	71.13	<u>5.23</u>
		FlexRound	<u>53.45</u>	<u>6.41</u>	<u>50.22</u>	<u>10.11</u>	<u>72.44</u>	5.81
	BCQ	ALTERNATING	<u>24.19</u>	1.1e4	<u>25.51</u>	8.4e4	<u>24.06</u>	<u>1.5e3</u>
		UniQuan_F	53.68	6.20	53.46	8.75	74.79	4.24

[†] UQ and BCQ methods have the same costs when using BCQ kernels (Park et al., 2024)

Table 4: Accuracies (%) of quantized Llama-3 Instruct 8B models on GSM8K benchmark. UniQuan_F outperforms the competitors in all cases.

Scheme	Method	4bit	3bit
Full precision		76.12	
UQ	RTN	49.43	0.08
	OmniQuant	59.16	2.78
	FlexRound	<u>70.66</u>	<u>54.13</u>
BCQ	ALTERNATING	0.00	0.00
	UniQuan_F	72.38	58.73

GSM8K (Cobbe et al., 2021). We sample 128 token sequences of length 2048 from C4 (Raffel et al., 2020) and GSM8K (Cobbe et al., 2021) for general and task-specific benchmarks, respectively. All experiments are done with a single A100 GPU.

Baselines. We compare UniQuan_F with the four UQ and BCQ methods: RTN (Gupta et al., 2015), OmniQuant (Shao et al., 2024), FlexRound (Lee et al., 2023), and ALTERNATING (Xu et al., 2018).

Hyperparameters. We set each row of weight matrices as a single weight group except for Llama-3 70B where we divide each row into small weight groups of size 128 to prevent severe accuracy loss. Note that each row has at least 1024 weights. Hyperparameters of UniQuan_F is in Appendix C.1.

4.2 General Knowledge Evaluation

We evaluate the amount of general knowledge retained in quantized models. Table 3 summarizes the average accuracies of 5-shot and 0-shot MMLU,

and perplexity on WikiText2 (Wiki). UniQuan_F achieves the highest performance in almost all cases, outperforming the second-best method by up to 3.24% in average MMLU accuracy. In Llama-3 70B, a small group size of 128 makes a small number of weights sharing their quantization parameters, reducing the difficulty of quantization; competitors show comparative performance to UniQuan_F in both 3- and 4-bit cases.

4.3 Task-specific Knowledge Evaluation

Table 4 summarizes the accuracies of a quantized Llama-3 8B Instruct model evaluated on GSM8K benchmark, which focuses exclusively on mathematical problems. Note that GSM8K is a difficult task in which models should generate the exact answer, while general knowledge benchmarks require only selecting correct answers among multiple choices, or evaluating given texts. UniQuan_F achieves the highest performance, outperforming the second-best competitor by 4.60% and 1.72% in the 3-bit and 4-bit experiments, respectively. Notably, UniQuan_F significantly improves the performance of FlexRound by utilizing ALTERNATING even when ALTERNATING exhibits zero accuracy. This demonstrates that UniQuan_F effectively unifies UQ and BCQ methods, leading to achieving previously unattainable performance.

4.4 Ablation Study

Table 5 summarizes the result of an ablation study using a Llama-3 8B model. We compare the three

Table 5: Ablation study results on MMLU benchmark. All ideas contribute to improving accuracies.

Method	4bit	3bit
UniQuan_F	61.43	53.46
- remapping	56.89	47.83
- unified init. (Θ_F)	24.70	22.87
- unified init. (Θ_B)	60.84	52.14

variants of UniQuan: “-remapping” without local and periodic mapping, “-unified init. (Θ_F)” initializing $\{\Delta, z_U, s, s_r\} \in \Theta_F$ as $\{1, 0, 1_g, 1\}$, and “-unified init. (Θ_B)” initializing α and z_B in Θ_B to have the uniformly spaced quantization levels. Note that UniQuan_F outperforms all of its variants, showing the effectiveness of our main ideas. Specifically, precisely initializing Θ_F is crucial to exploit the strong optimizability of Θ_F .

4.5 Case Study

We conduct a case study on a Llama-3 8B model quantized with UniQuan_F to verify whether the intended mechanisms function properly.

Expressiveness. Figure 4(a) visualizes the distribution of weights within a weight group and the assigned quantization levels following UniQuan_F. As shown in the figure, UniQuan_F assigns dense quantization levels near zero, where most weights are concentrated, and sparse levels in regions with fewer weights. This demonstrates that UniQuan_F effectively learns non-linear quantization levels aligned to the weight distribution, harnessing the strong expressiveness of BCQ scheme.

Optimizability. Figure 4(b) shows the changes of reconstruction error during optimization for a block when using FlexRound, ALTERNATING*, and UniQuan_F; ALTERNATING* is a modification of ALTERNATING that optimizes α using a block-wise output reconstruction process in UniQuan_F devised for comparing optimizability. Compared to the baselines, UniQuan_F achieves the least error at the end with a steep decrease at the early stage of optimization. This demonstrates that UniQuan_F effectively minimizes the reconstruction error, harnessing the strong optimizability of UQ scheme.

In summary, UniQuan_F successfully unifies UQ and BCQ schemes, harnessing their strong expressiveness and optimizability, as we intended. We further verify that UniQuan_F effectively leverages flexible mapping, the main optimization technique of FlexRound, in Appendix F.6.

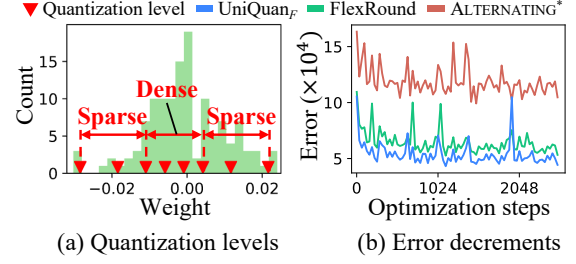


Figure 4: (a) Quantization levels learned by UniQuan_F and (b) changes of reconstruction error during optimization. UniQuan_F assigns quantization levels that align closely with the weight distribution, and effectively reduces the reconstruction error during optimization.

5 Related Work

Quantization (Lin et al., 2024; Ashkboos et al., 2024; Liu et al., 2024b) has gained great attention since it effectively speeds up LLM inference by minimizing the cost of memory operations. The majority of quantization approaches rely on uniform quantization (UQ) (Shao et al., 2024; Lee et al., 2023) due to its hardware-friendly nature, benefiting from pre-existing acceleration kernels. Some works (Ashkboos et al., 2024; Liu et al., 2024b) even quantize activation by mitigating outliers through weight rotation, which is orthogonal to our approach. Other approaches, such as vector (Kim et al., 2024; Tseng et al., 2024; Liu et al., 2024a) and additive quantization (Egiazarian et al., 2024) suffer from slow inference since they lack of acceleration kernels. Binary-coding quantization (BCQ) (Xu et al., 2018; Kwon et al., 2022) also has been underexplored due to its missing kernels even with broader expressiveness over UQ (see Section 2.3). However, recent studies (Park et al., 2024; You et al., 2024) have proposed kernels leveraging look-up tables, enabling BCQ to achieve the same speed as UQ. Taking advantage of them, UniQuan unlocks the superior expressiveness of BCQ by unifying it with the UQ scheme.

6 Conclusion

We propose UniQuan_F, an accurate quantization method for LLMs. We unify the best-performing uniform quantization (UQ) and binary-coding quantization (BCQ) methods to leverage their strong optimizability and expressiveness concurrently. We propose local and periodic mapping, unified initialization, and composition theorem to effectively optimize and deploy the quantized models. As a result, UniQuan_F achieves the best performance, demonstrating the effectiveness of the unification.

Limitations

In this paper, we define UniQuan which is a quantization scheme that unifies uniform quantization (UQ) and binary-coding quantization (BCQ) schemes. We verify the effectiveness of UniQuan with only UniQuan_F which unifies the best-performing UQ and BCQ methods. We cover the unification of extended quantization methods in Appendix E.6 in theoretical aspects. The experimental validation of diverse pairs of quantization methods in extended quantization schemes is one of our promising future works.

References

- Saleh Ashkboos, Amirkeivan Mohtashami, Maximilian L. Croci, Bo Li, Martin Jaggi, Dan Alistarh, Torsten Hoefer, and James Hensman. 2024. Quarot: Outlier-free 4-bit inference in rotated llms. *CoRR*, abs/2404.00456.
- Yoshua Bengio, Nicholas Léonard, and Aaron C. Courville. 2013. Estimating or propagating gradients through stochastic neurons for conditional computation. *CoRR*, abs/1308.3432.
- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.
- Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022. Gpt3. int8 (): 8-bit matrix multiplication for transformers at scale. *Advances in Neural Information Processing Systems*, 35:30318–30332.
- Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.
- Vage Egiazarian, Andrei Panferov, Denis Kuznedelev, Elias Frantar, Artem Babenko, and Dan Alistarh. 2024. Extreme compression of large language models via additive quantization. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*. OpenReview.net.
- Elias Frantar and Dan Alistarh. 2023. Sparsegpt: Massive language models can be accurately pruned in one-shot. In *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA*, volume 202 of *Proceedings of Machine Learning Research*, pages 10323–10337. PMLR.
- Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. 2023. OPTQ: Accurate quantization for generative pre-trained transformers. In *The Eleventh International Conference on Learning Representations*.
- Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac’h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. 2023. A framework for few-shot language model evaluation.
- Suyog Gupta, Ankur Agrawal, Kailash Gopalakrishnan, and Pritish Narayanan. 2015. Deep learning with limited numerical precision. In *International conference on machine learning*, pages 1737–1746. PMLR.
- Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021. Measuring massive multitask language understanding. *Proceedings of the International Conference on Learning Representations (ICLR)*.
- Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de Las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Léo Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2023. Mistral 7b. *CoRR*, abs/2310.06825.
- Sehoon Kim, Coleman Richard Charles Hooper, Amir Gholami, Zhen Dong, Xiuyu Li, Sheng Shen, Michael W Mahoney, and Kurt Keutzer. 2024. Squeezellm: Dense-and-sparse quantization. In *Forty-first International Conference on Machine Learning*.
- Se Jung Kwon, Jeonghoon Kim, Jeongin Bae, Kang Min Yoo, Jin-Hwa Kim, Baeseong Park, Byeongwook Kim, Jung-Woo Ha, Nako Sung, and Dongsoo Lee. 2022. Alphasatuning: Quantization-aware parameter-efficient adaptation of large-scale pre-trained language models. In *Findings of the Association for Computational Linguistics: EMNLP 2022, Abu Dhabi, United Arab Emirates, December 7-11, 2022*, pages 3288–3305. Association for Computational Linguistics.
- Jung Hyun Lee, Jeonghoon Kim, Se Jung Kwon, and Dongsoo Lee. 2023. Flexround: Learnable rounding based on element-wise division for post-training quantization. In *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA*, volume 202 of *Proceedings of*

703	<i>Machine Learning Research</i> , pages 18913–18939.	
704	PMLR.	
705	Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-	
706	Ming Chen, Wei-Chen Wang, Guangxuan Xiao,	
707	Xingyu Dang, Chuang Gan, and Song Han. 2024.	
708	Awq: Activation-aware weight quantization for on-	
709	device llm compression and acceleration. <i>Proceed-</i>	
710	<i>ings of Machine Learning and Systems</i> , 6:87–100.	
711	Yifei Liu, Jicheng Wen, Yang Wang, Shengyu Ye,	
712	Li Lyna Zhang, Ting Cao, Cheng Li, and Mao Yang.	
713	2024a. VPTQ: extreme low-bit vector post-training	
714	quantization for large language models. In <i>Proceed-</i>	
715	<i>ings of the 2024 Conference on Empirical Methods in</i>	
716	<i>Natural Language Processing, EMNLP 2024, Miami,</i>	
717	<i>FL, USA, November 12-16, 2024</i> , pages 8181–8196.	
718	Association for Computational Linguistics.	
719	Zechun Liu, Changsheng Zhao, Igor Fedorov, Bilge	
720	Soran, Dhruv Choudhary, Raghuraman Krishnamoor-	
721	thi, Vikas Chandra, Yuandong Tian, and Tijmen	
722	Blankevoort. 2024b. Spinquant: LLM quantization	
723	with learned rotations . <i>CoRR</i> , abs/2405.16406.	
724	Stephen Merity, Caiming Xiong, James Bradbury, and	
725	Richard Socher. 2017. Pointer sentinel mixture mod-	
726	els. In <i>5th International Conference on Learning</i>	
727	<i>Representations, ICLR 2017, Toulon, France, April</i>	
728	<i>24-26, 2017, Conference Track Proceedings</i> . Open-	
729	Review.net.	
730	Gunho Park, Baeseong Park, Minsub Kim, Sungjae Lee,	
731	Jeonghoon Kim, Beomseok Kwon, Se Jung Kwon,	
732	Byeongwook Kim, Youngjoo Lee, and Dongsoo Lee.	
733	2024. LUT-GEMM: quantized matrix multiplication	
734	based on luts for efficient inference in large-scale	
735	generative language models. In <i>The Twelfth Inter-</i>	
736	<i>national Conference on Learning Representations,</i>	
737	<i>ICLR 2024, Vienna, Austria, May 7-11, 2024</i> . Open-	
738	Review.net.	
739	Adam Paszke, Sam Gross, Francisco Massa, Adam	
740	Lerer, James Bradbury, Gregory Chanan, Trevor	
741	Killeen, Zeming Lin, Natalia Gimelshein, Luca	
742	Antiga, et al. 2019. Pytorch: An imperative style,	
743	high-performance deep learning library. <i>Advances in</i>	
744	<i>neural information processing systems</i> , 32.	
745	Colin Raffel, Noam Shazeer, Adam Roberts, Katherine	
746	Lee, Sharan Narang, Michael Matena, Yanqi Zhou,	
747	Wei Li, and Peter J. Liu. 2020. Exploring the limits	
748	of transfer learning with a unified text-to-text trans-	
749	former. <i>J. Mach. Learn. Res.</i> , 21:140:1–140:67.	
750	Wenqi Shao, Mengzhao Chen, Zhaoyang Zhang, Peng	
751	Xu, Lirui Zhao, Zhiqian Li, Kaipeng Zhang, Peng	
752	Gao, Yu Qiao, and Ping Luo. 2024. Omniquant:	
753	Omnidirectionally calibrated quantization for large	
754	language models. In <i>The Twelfth International Con-</i>	
755	<i>ference on Learning Representations, ICLR 2024,</i>	
756	<i>Vienna, Austria, May 7-11, 2024</i> . OpenReview.net.	
757	Hugo Touvron, Louis Martin, Kevin Stone, Peter Al-	
758	bert, Amjad Almahairi, Yasmine Babaei, Nikolay	
759	Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti	
	Bhosale, et al. 2023. Llama 2: Open founda-	760
	tion and fine-tuned chat models. <i>arXiv preprint</i>	761
	<i>arXiv:2307.09288</i> .	762
	Albert Tseng, Jerry Chee, Qingyao Sun, Volodymyr	763
	Kuleshov, and Christopher De Sa. 2024. Quip#:	764
	Even better LLM quantization with hadamard inco-	765
	herence and lattice codebooks. In <i>Forty-first Interna-</i>	766
	<i>tional Conference on Machine Learning, ICML 2024,</i>	767
	<i>Vienna, Austria, July 21-27, 2024</i> . OpenReview.net.	768
	Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob	769
	Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz	770
	Kaiser, and Illia Polosukhin. 2017. Attention is all	771
	you need. <i>Advances in neural information processing</i>	772
	<i>systems</i> , 30.	773
	Thomas Wolf, Lysandre Debut, Victor Sanh, Julien	774
	Chaumond, Clement Delangue, Anthony Moi, Pier-	775
	ric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz,	776
	and Jamie Brew. 2019. Huggingface’s transformers:	777
	State-of-the-art natural language processing . <i>CoRR</i> ,	778
	abs/1910.03771.	779
	Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu,	780
	Julien Demouth, and Song Han. 2023. Smoothquant:	781
	Accurate and efficient post-training quantization for	782
	large language models. In <i>International Conference</i>	783
	<i>on Machine Learning</i> , pages 38087–38099. PMLR.	784
	Chen Xu, Jianqiang Yao, Zhouchen Lin, Wenwu Ou,	785
	Yuanbin Cao, Zhirong Wang, and Hongbin Zha. 2018.	786
	Alternating multi-bit quantization for recurrent neu-	787
	ral networks . In <i>6th International Conference on</i>	788
	<i>Learning Representations, ICLR 2018, Vancouver,</i>	789
	<i>BC, Canada, April 30 - May 3, 2018, Conference</i>	790
	<i>Track Proceedings</i> . OpenReview.net.	791
	Haoran You, Yipin Guo, Yichao Fu, Wei Zhou, Hui-	792
	hong Shi, Xiaofan Zhang, Souvik Kundu, Amir	793
	Yazdanbakhsh, and Yingyan Celine Lin. 2024.	794
	Shiftaddllm: Accelerating pretrained llms via	795
	post-training multiplication-less reparameterization.	796
	<i>arXiv preprint arXiv:2405.04532</i> .	797
	Susan Zhang, Stephen Roller, Naman Goyal, Mikel	798
	Artetxe, Moya Chen, Shuohui Chen, Christopher De-	799
	wan, Mona Diab, Xian Li, Xi Victoria Lin, et al. 2022.	800
	Opt: Open pre-trained transformer language models.	801
	<i>arXiv preprint arXiv:2205.01068</i> .	802
	A Terminology	803
	We summarize the definitions of terminologies fre-	804
	quently used in this paper to promote clarity.	805
	A.1 Units in LLMs	806
	We summarize the definitions of the units in Large	807
	Language Models (LLMs) from a weight to a	808
	model. Figure 5 illustrates an example of a	809
	Transformer-based LLM with N blocks.	810

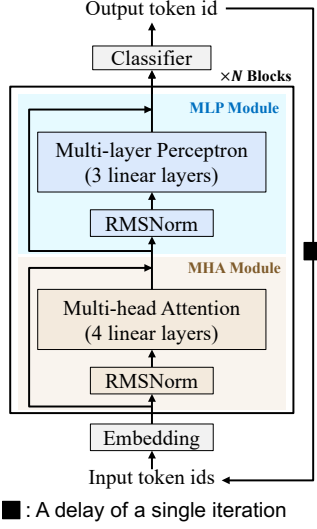


Figure 5: An illustration of a Transformer architecture with L blocks.

- **Weight:** the smallest unit, representing an individual numerical weight value. The quantization bit width represents the number of bits to represent each weight.
- **Weight group:** a collection of weights grouped by a specified group size, all of which share the same quantization parameters. It is easy to quantize models when we have small group sizes since we have plenty of quantization parameters.
- **Weight matrix:** a two-dimensional matrix composed of weights, containing multiple weight groups.
- **Layer:** a component that performs affine transformations with a weight matrix and a bias vector.
- **Module:** a collection of layers that performs a specific functionality. In Transformers, modules include Multi-Head Attention (MHA) and Multi-Layer Perceptron (MLP).
- **Block:** a fundamental unit of a Transformer, consisting of one MHA module and one MLP module. FlexRound (Lee et al., 2023) and UniQuan_F sequentially quantize each block from the bottom to top to reduce the cost of quantization.
- **Model:** a complete language model consisting of multiple blocks. An LLM refers to a model.

A.2 Error Types

In this paper, we mention two types of errors: quantization error and output reconstruction error. Quantization error represents the error before and after quantization at the weight level without considering the model’s input. If we quantize a weight group

Table 6: Symbols and their definitions.

Symbol	Definition
w	Weight
$\mathbf{w}$	Group of weights
$\tilde{\mathbf{w}}$	Group of transformed weights
$\hat{\mathbf{w}}$	Group of mapped weights
Φ	Set of weight groups
g	The size of a weight group
k	Quantization bit-width
f	Transformer block
$ \mathbb{D} $	Number of data points
$\mathbf{1}_g$	Vector of size g filled with ones
$\lfloor \cdot \rfloor$	Rounding function
$\text{Clip}(\cdot, m, M)$	Clipping function with range $[m, M]$
$\oslash$	Element-wise division
Δ	Scale factor of UQ
z_U	Zero-point of UQ
$\mathbf{C}$	Binary-code matrix of BCQ
α	Scale factors of BCQ
z_B	Shifting factor of BCQ
s	Element-wise scale factor of FlexRound
s_r	Row-wise scale factor of FlexRound
G	Grid search iterations
T	Alternating update iterations
p	Remapping period

w into $\hat{w}$, then the quantization error is $\|w - \hat{w}\|_2^2$. On the other hand, output reconstruction error measures how the quantized model’s outputs are close to those before quantization. We measure the reconstruction error at block level. Assume that we have a block f with parameters Φ , quantized into $\hat{f}$ with parameters $\hat{\Phi}$. The reconstruction error is $\|f(\mathbf{X}; \Phi) - \hat{f}(\mathbf{X}; \hat{\Phi})\|_F^2$, where $\mathbf{X}$ and $\hat{\mathbf{X}}$ are the inputs of the block f and the quantized block $\hat{f}$, respectively.

We first initialize the quantization parameters to minimize quantization errors at the weight level. Then, we optimize the quantization parameters to minimize the output reconstruction errors at the block level to take account of the input distribution.

A.3 Symbols and Definitions

We summarize the frequently used symbols and their definitions in Tables 6 and 7.

B Inference Speed of BCQ Kernels

In this paper, we propose UniQuan which unifies UQ and BCQ schemes, and the resulting models are represented in a BCQ scheme. The effectiveness of UniQuan depends on the inference speed of BCQ kernels (Park et al., 2024; You et al., 2024). Thus, we summarize the inference speed of LUT-GEMM (Park et al., 2024), the state-of-the-art BCQ kernel, for completeness.

Table 7: A summary of terminologies regarding different quantization methods. $\Rightarrow$ represents the application of the unification theorem (see Theorem 1). UniQuan’s quantization parameters Θ_U , transformation function $\mathcal{T}_U$ and detransformation function $\mathcal{D}_U$ are determined according to the UQ method unified into the BCQ scheme.

Method	Quantizer	Quantization Parameters	Transformation Function	Mapping Function	Detransformation Function	Reconstruction Function
General form	Q	Θ	$\mathcal{T}(w; \Theta)$	$\mathcal{M}(w; \Theta)$	$\mathcal{D}(w; \Theta)$	$\mathcal{R}_B(C; \Theta)$
RTN	Q_R	$\Theta_R = \{\Delta, z_U\}$	$\mathcal{T}_R(w; \Theta_R)$	$\mathcal{M}_U(w, k)$	$\mathcal{D}_R(w; \Theta_R)$	-
FlexRound	Q_F	$\Theta_F = \{\Delta, z_U, s, s_r\}$	$\mathcal{T}_F(w; \Theta_F)$			
ALTERNATING	Q_B	$\Theta_B = \{\alpha, z_B\}$	$\mathcal{T}_B(w)$	$\mathcal{M}_B(w; \Theta_B)$	$\mathcal{D}_B(w)$	$\mathcal{R}_B(C; \Theta_B)$
UniQuan	Q_I	$\Theta_I = \Theta_U \cup \Theta_B \Rightarrow \Theta_B^*$	$\mathcal{T}_U(w; \Theta_U)$	$\mathcal{M}_B^*(w; \Theta_B)$	$\mathcal{D}_U(w; \Theta_U)$	$\mathcal{R}_B(C; \Theta_B^*)$
UniQuan _F	Q_{I_F}	$\Theta_{I_F} = \Theta_F \cup \Theta_B \Rightarrow \Theta_B^*$	$\mathcal{T}_F(w; \Theta_F)$		$\mathcal{D}_F(w; \Theta_F)$	

Table 8: Latency comparison of the first FFN layer on OPT-175B model with various precision and kernels on A100-80GB-GPU.

Kernel	Schemes	# Bits	Latency (ms)
cuBLAS	-	16	0.7256
GPTQ	UQ	3	0.3599
AWQ	UQ	4	0.3238
LUT-GEMM	UQ, BCQ	4	0.2688
LUT-GEMM	UQ, BCQ	3	0.2250

Table 9: Comparison of end-to-end latency per token for OPT-30B models on a A100-80GB-GPU.

Model	Kernel- k - g	Latency (ms)
OPT-30B	cuBLAS-16-N/A	40.5
	LUT-GEMM-4-32	18.5
	LUT-GEMM-4-64	17.8
	LUT-GEMM-3-32	16.7
	LUT-GEMM-3-64	15.7

Table 8 summarizes the main results of Table 1 in LUT-GEMM (Park et al., 2024). In this table, latency represents the time for inferencing the first FFN layer in OPT-175B (Zhang et al., 2022) with various precision, and LUT-GEMM shows faster inference speeds than GPTQ (Frantar et al., 2023) and AWQ (Lin et al., 2024) kernels, which support only a UQ scheme.

Table 9 summarizes the main results of Table 6 in LUT-GEMM (Park et al., 2024), reporting the end-to-end latency per token for OPT family models. k and g represent the bit width and group size, respectively. As summarized in the table, LUT-GEMM provides end-to-end inference speedup with diverse bit-widths and group sizes.

In summary, LUT-GEMM provides faster inference speed for quantized models than existing UQ kernels and speeds up end-to-end infer-

ence. Therefore, UniQuan_F, which is supported by LUT-GEMM, is an essential method for quantizing LLMs. Note that we need to convert quantized models using a UQ scheme into a BCQ scheme to use LUT-GEMM which provides a faster inference speed than existing UQ kernels as shown in Table 8. Therefore, quantized models in both schemes require the same memory and computational costs when they have the same bit width.

C Implementation Details

We use PyTorch (Paszke et al., 2019) and Hugging Face (Wolf et al., 2019) libraries for implementation. We use the pretrained weights of Llama-3 (Dubey et al., 2024) and Mistral (Jiang et al., 2023) models from the Hugging Face library. We discuss the implementation details of UniQuan_F and competitors to reproduce the performance reported in this paper.

C.1 Implementation Details of UniQuan_F

Hyperparameter settings. Our objective is to demonstrate that the outstanding performance of UniQuan_F is achieved without expensive hyperparameter tuning although UniQuan_F employs a block-wise output reconstruction process that requires many hyperparameters. To this end, we fix all hyperparameters except for the grid size G for unified initialization and utilize only two combinations of hyperparameters across all cases. We outline these combinations in Table 10.

Gradient filtering. We utilize a straight-through estimator (STE) (Bengio et al., 2013) to update UniQuan_F’s quantization parameters since its mapping function $\mathcal{M}_B^*$ is not differentiable. We filter the gradient of the weights that have large mapping errors $|\bar{w} - \tilde{w}|$ to stabilize the optimization process of UniQuan_F, where $\bar{w}$ and $\tilde{w}$ represent trans-

Table 10: Hyperparameter settings of UniQuan_F

Hyperparameter	Setting
Learning rate for Θ_F	0.005
Learning rate for Θ_B	0.0005
Grid search iterations (G)	0, 30
Alternating update iterations (T)	15
Remapping Period (p)	2
Epochs	20
Batch size	1
Clipping strategy	Fixed-minimum

formed and mapped weights, respectively. STE hypothesizes that the gradient of a transformed weight $\bar{w}$ and mapped weight $\tilde{w}$ have the same gradients. However, if the difference between transformed and mapped weight is significant, the hypothesis does not hold which degrades the accuracy of quantized models. Therefore, we set the hyperparameter τ as a gradient filtering threshold, and zero out the gradients of weights whose mapping error is larger than τ .

We mimic the quantization process in the UQ scheme to determine τ . In UQ, weights within the clipping range $[w_{m,c}, w_{M,c}]$ are transformed to the values in the range $[0, 2^{k-1}]$, while weights outside the clipping range, such as w_m and w_M , are transformed into values outside the range $[0, 2^{k-1}]$. These out-of-range weights cause significant mapping errors. In the transformed space, the interval between quantization levels is 1, and each level has a range of 0.5 on either side. Thus, it is reasonable to allow a margin of 0.5 even for the smallest and largest quantization levels 0 and 2^{k-1} .

For UniQuan_F, every quantization level has a potential risk of significant mapping errors, and the value of 0.5 is replaced with the smallest scale factor $\min(\alpha)$ which is the smallest width in the binary tree constructed by BCQ’s quantization levels (see Figure 2 (b)). Therefore, we set the gradient filtering threshold τ as $\min(\alpha)$.

C.2 Implementation Details of Competitors

RTN (Gupta et al., 2015). We search the clipping range with 100 iterations, which is larger than the number of iterations used in unified initialization. We execute the same source code for clipping range search in UniQuan_F.

ALTERNATING (Xu et al., 2018). We implement ALTERNATING based on the original paper (Xu et al., 2018). We use an alternating update with 15 iterations which is the same as the number

of iterations used in unified initialization. We use the same source code for the alternating update in UniQuan_F.

FlexRound (Lee et al., 2023). We implement the FlexRound following the original paper (Lee et al., 2023).

OmniQuant (Shao et al., 2024). We refer to the official implementation² of OmniQuant and report the results following the best hyperparameter settings reported in the paper.

D Evaluation Protocol

We report the average performance using random seeds 0, 1, and 2 except for Llama-3 70B which uses only 0 because of its long quantization time. We sample 128 token sequences of length 2048 from C4 (Raffel et al., 2020) and GSM8K (Cobbe et al., 2021) for general and task-specific knowledge evaluations, respectively. We detail the evaluation protocols for general and task-specific knowledge evaluation as follows.

General knowledge evaluation. We use MMLU (Hendrycks et al., 2021) and WikiText2 (Merity et al., 2017) benchmarks for general knowledge evaluation. MMLU consists of multiple-choice problems across 57 subjects. We evaluate the 0-shot and 5-shot accuracies on MMLU to evaluate the amount of general knowledge in the quantized models. 0-shot and 5-shot refer to settings with 0 and 5 examples provided in the prompt, respectively. We follow the evaluation protocol in the official code repository¹ for the MMLU benchmark. WikiText2 consists of tokens within a set of verified articles from Wikipedia. We report the perplexity of quantized models on WikiText2 benchmark to evaluate the language modeling capabilities of quantized models. We follow the evaluation protocol used in OmniQuant (Shao et al., 2024) according to its official implementation².

Task-specific knowledge evaluation. We use GSM8K (Cobbe et al., 2021) benchmark for task-specific knowledge evaluation. GSM8K consists of high-quality grade school math problems that require multi-step reasoning, and we use GSM8K to evaluate the amount of Mathematical knowledge in the quantized models. We implement our evaluation code using the language model evaluation harness (Gao et al., 2023) package³.

¹<https://github.com/hendrycks/test>

²<https://github.com/OpenGVLab/OmniQuant/tree/main>

³<https://github.com/EleutherAI/>

Algorithm 3 General alternating update

Input: A weight group $\mathbf{w}$ of size g . Numbers G and T of iterations for grid search and alternating update, respectively

Output: Updated α and z_B

- 1: Greedy initialize α and $\mathbf{C}$
 $\triangleright$ Equation 4 in Xu et al. (2018)
 - 2: **for** t in $1, 2, \dots, T$ **do**
 - 3: $\alpha \leftarrow (\mathbf{C}^T \mathbf{C})^{-1} \mathbf{C}^T (\mathbf{w} - z_B)$
 - 4: $\mathbf{C} \leftarrow \arg \min_{\mathbf{C}'} \|\mathbf{w} - (\mathbf{C}'^T \alpha + z_B)\|_2^2$
 - 5: **if** $G = 0$ **then**
 $\triangleright$ Update z_B only if not grid search
 - 6: $z_B \leftarrow \sum_{i=1}^g (w_i - \mathbf{C}_i^T \alpha) / g$
 - 7: **end if**
 - 8: **end for**
-

E.3 Algorithm of General Alternating Update

Algorithm 3 details the general alternating update process employed in line 6 of Algorithm 2 which is an improved version of alternating update in Xu et al. (2018) to find proper z_B . We initialize α and $\mathbf{C}$ using a greedy initialization strategy (Equation (4) in Xu et al. (2018)), where we determine α and $\mathbf{C}$ for each bit-width one by one (line 1). Subsequently, we iteratively minimize the quantization error $e = \|\mathbf{w} - (\mathbf{C}'^T \alpha + z_B)\|_2^2$ for T iterations to find the optimal quantization parameters (lines 2-8). In each iteration, we optimize for α , $\mathbf{C}$, and z_B sequentially while keeping the others fixed. For α and z_B , we find the values where the derivatives $\frac{\partial e}{\partial \alpha}$ and $\frac{\partial e}{\partial z_B}$ equal to zero, respectively (lines 3 and 6). For $\mathbf{C}$, we find the optimal binary codes for each weight by comparing errors obtained using all possible binary codes (line 4). After completing these iterations, we obtain the optimized values for α and z_B . We do not update z_B when G is greater than zero since diverse values for z_B are explored during the iterative grid search process (line 5).

E.4 Algorithm of Local and Periodic Mapping

Algorithm 4 describes the overall process of local and periodic mapping. In the first optimization step, we initialize the mapping by calculating distances to all quantization levels. Then, we begin local and periodic mapping using the resulting mapping $\mathbf{d}$ and mapped weights $\tilde{\mathbf{w}}$ in the second step where $\mathbf{d}$ contains the indices of quantization levels that the weights are mapped. We periodically update the mappings between weights and quantization levels in every p step (line 1). We compute

Algorithm 4 Local and periodic mapping ($\mathcal{M}_B^*$)

Input: A transformed weight group $\tilde{\mathbf{w}}$ of size g , iteration step s , a remapping period p , and BCQ's quantization parameter $\Theta_B = \{\alpha, z_B\}$. Mapping indices $\mathbf{d}$ and mapped weights $\tilde{\mathbf{w}}$ in the previous step

Output: Updated mapping indices $\mathbf{d}$ and mapped weights $\tilde{\mathbf{w}}$

- 1: **if** $s \equiv 0 \pmod{p}$ **then** $\triangleright$ Periodic mapping
 - 2: $\mathbf{q} \leftarrow \text{compute-quantization-levels}(\alpha, z_B)$
 - 3: **for** i in $\{1, 2, \dots, g\}$ **do**
 - 4: $l, r \leftarrow \max(d_i - 1, 1), \min(d_i + 1, 2^k)$
 - 5: $\mathcal{N} \leftarrow \{\mathbf{q}_l, \mathbf{q}_{d_i}, \mathbf{q}_r\}$
 $\triangleright$ Neighbor quantization levels
 - 6: $d_i \leftarrow \arg \min_{q \in \mathcal{N}} |\tilde{w}_i - q|$
 $\triangleright$ Local mapping
 - 7: $\tilde{w}_i \leftarrow q_{d_i}$
 - 8: **end for**
 - 9: **end if**
-

quantization levels using α and z_B , and store them in $\mathbf{q} \in \mathbb{R}^{2^k}$ (line 2). Then, we find the closest quantization level among its previously mapped level and its neighboring levels (lines 4-6). We also update the mapped weight value to use them for the successive steps before the next update (line 7). Local and periodic mapping is efficient, updating $1/p$ times less often than non-periodic algorithms and using only 3 among 2^k quantization levels for computing the distance between weights.

E.5 Proof of Theorem 1

Proof. By the definitions of $\mathcal{R}_B(\mathbf{C}; \Theta_B)$ and $\mathcal{D}_R(\tilde{\mathbf{w}}; \Theta_R)$, we reduce $\mathcal{D}_R(\mathcal{R}_B(\mathbf{C}; \Theta_B); \Theta_R)$ as follows.

$$\begin{aligned} \mathcal{D}_R(\mathcal{R}_B(\mathbf{C}; \Theta_B); \Theta_R) &= \Delta((\mathbf{C}\alpha + z_B \mathbf{1}_g) - z_U \mathbf{1}_g) \\ &= \mathbf{C}(\Delta\alpha) + \Delta(z_B - z_U) \mathbf{1}_g. \end{aligned} \tag{1126}$$

Thus, $\alpha^* = \Delta\alpha$, $z_B^* = \Delta(z_B - z_U)$, and $\mathcal{R}_B(\mathbf{C}; \Theta_B^*) = \mathbf{C}\alpha^* + z_B^* \mathbf{1}_g$ where $\Theta_B^* = \{\alpha^*, z_B^*\}$. $\square$

E.6 Extensibility of UniQuan

In the main text, we employ UniQuan_F which unifies FlexRound (Lee et al., 2023) and ALTERNATING, the best-performing UQ and BCQ methods. We cover the unification between other quantization methods to explain the extensibility of UniQuan. The main idea of UniQuan is to utilize the unified quantization process with quantization pa-

parameters $\Theta_I = \Theta_U \cup \Theta_B$ as follows:

$$Q_I(\mathbf{w}; \Theta_I) = \mathcal{D}_U(\mathcal{M}_B(\mathcal{T}_U(\mathbf{w}; \Theta_U); \Theta_B); \Theta_U).$$

Considering the lack of accurate BCQ methods, we focus on replacing $\mathcal{T}_U$ and $\mathcal{D}_U$ with method-specific ones to unify UQ methods into the BCQ scheme.

AWQ (Lin et al., 2024) reduces output reconstruction error by scaling weights based on their importance before quantization. This is accomplished by using $\mathcal{T}_A$ and $\mathcal{D}_A$ defined as follows:

$$\begin{aligned}\mathcal{T}_A(\mathbf{w}; \Theta_A) &= (\mathbf{w} \odot \mathbf{s}_A) / \Delta_A + z_A \mathbf{1}_g, \\ \mathcal{D}_A(\tilde{\mathbf{w}}; \Theta_A) &= \Delta_A(\tilde{\mathbf{w}} - z_O \mathbf{1}_g) \odot \mathbf{s}_A,\end{aligned}$$

where $\mathbf{s}_A \in \mathbb{R}^g$ is a column-wise scale factor to reflect the importance of each column of the weight matrix where the weight group $\mathbf{w}$ is located. $\Delta_A \in \mathbb{R}$ and $z_A \in \mathbb{R}$ represent the new scale factor and zero-point after applying $\mathbf{s}_A$, respectively. $\Theta_A = \{\Delta_A, z_A, \mathbf{s}_A\}$ is a set of quantization parameters for AWQ. $\odot$ and $\oslash$ represent element-wise multiplication and division, respectively.

OmniQuant (Shao et al., 2024) parameterizes the clipping range with parameters β and γ to find the precise clipping range through optimization. This is accomplished by using $\mathcal{T}_O$ and $\mathcal{D}_O$ as follows:

$$\begin{aligned}\mathcal{T}_O(\mathbf{w}; \Theta_O) &= \mathbf{w} / \Delta_O + z_O \mathbf{1}_g, \\ \mathcal{D}_O(\tilde{\mathbf{w}}; \Theta_O) &= \Delta_O(\tilde{\mathbf{w}} - z_O \mathbf{1}_g),\end{aligned}$$

where $\Delta_O = (\gamma w_M - \beta w_m) / (2^k - 1)$ and $z_O = -\lfloor \beta w_m / \Delta_O \rfloor$ are the parameterized scale factor and zero-point, respectively. w_m and w_M are the minimum and maximum weights in a weight group $\mathbf{w}$. $\Theta_O = \{\beta, \gamma\}$ is the set of quantization parameters of OmniQuant.

GPTQ (Frantar et al., 2023) is widely used in a UQ scheme, but it is a general error compensation technique which covers even pruning (Frantar and Alistarh, 2023). Thus, GPTQ is able to be unified into a BCQ scheme by applying GPTQ’s error compensation strategy to BCQ methods.

In summary, it is able to unify diverse UQ methods into a BCQ scheme following UniQuan’s framework in theory. Extending the coverage of unification to other quantization schemes beyond UQ and BCQ, and validating with experiments is one of our promising future works.

F Additional Analyses

We analyze the memory usage, quantization time, hyperparameter sensitivity, and the effect of sample

dataset size on UniQuan_F. We also validate that UniQuan_F effectively leverages the optimization technique in FlexRound.

F.1 Memory Usage of UniQuan_F

We analyze the memory usage of quantized models using UniQuan_F. As summarized in Table 1, UniQuan_F requires the same amount of memory at inference time as existing BCQ methods (Xu et al., 2018; Kwon et al., 2022) since we convert the inference process of the unified quantization process into a single BCQ’s inference process using the unification theorem. When we quantize a weight group $\mathbf{w} \in \mathbb{R}^g$ of size g into k bits, we need to store a binary code matrix $\mathbf{C} \in \{-1, +1\}^{g \times k}$, BCQ’s scale factors $\alpha \in \mathbb{R}^k$, and BCQ’s shifting factor $z_B \in \mathbb{R}$. Each binary code requires 1 bit and each real number requires 16 bits for saving, thus we require $gk + 16(k + 1)$ bits in total. The memory overhead due to the quantization parameters is $16(k + 1)/g$ bits per weight, and if we assume channel-wise quantization where g is larger than 4000 for LLMs (Jiang et al., 2023; Dubey et al., 2024), the memory overhead is negligible.

F.2 Quantization Time of UniQuan_F

We compare the running time for quantizing Llama-3 8B into 3 bits and the accuracy of the quantized model, using UniQuan_F and FlexRound to evaluate the efficiency of UniQuan_F. We compare them across different epoch settings since the quantization time varies depending on the number of epochs. We do not include the time for unified initialization in UniQuan_F since it is performed as a preprocessing before quantization; the initialized quantization parameters are reused in different hyperparameter settings when the alternating update iteration T and the grid search iteration G are not changed.

Table 12 shows that UniQuan_F requires about 20% longer time for quantization than FlexRound, but it achieves higher accuracy than FlexRound across all epoch settings. Especially, UniQuan_F requires a shorter time of 15,905 seconds to outperform the highest accuracy of FlexRound which requires 16,868 seconds, demonstrating its efficiency.

The running time for the unified initialization process depends on G and T . When $G = 30$ and $T = 15$, as used in this experiment, it takes approximately 6,700 seconds and UniQuan_F takes longer than FlexRound if we include this. However, considering that the quantization time only needs to be performed once for deployment, the initialization

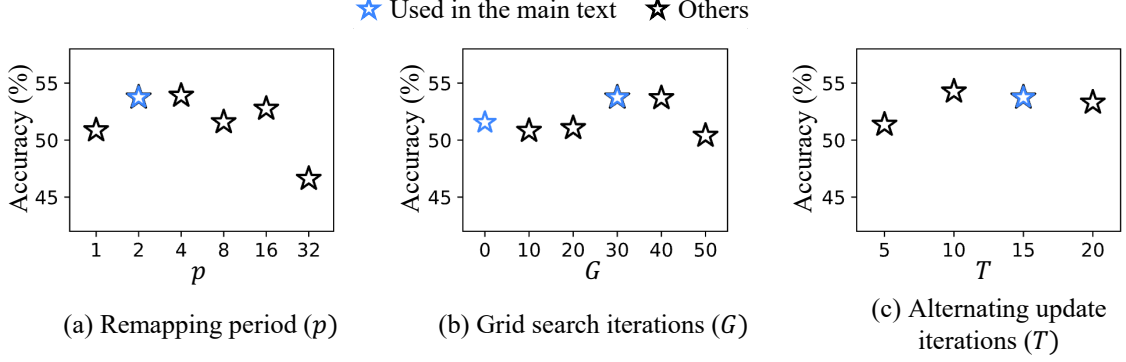


Figure 6: Change of average accuracy on 0-shot and 5-shot MMLU benchmarks with regard to the change of remapping period p , grid search iterations G , and alternating update iterations T . Blue stars represent the hyperparameters used in the main text and black stars represent the others.

Table 12: Comparison of average accuracies (%) on 0-shot and 5-shot MMLU benchmarks, and quantization time (s) when quantizing Llama-3 8B into 3bit using UniQuan_F and FlexRound, respectively. Bold and underlined texts represent the best and second-best results in each method, respectively.

Epochs	UniQuan _F		FlexRound	
	Accuracy	Time	Accuracy	Time
5	47.97	5,089	46.71	4,239
10	51.04	10,022	47.97	8,350
15	52.08	15,905	51.08	12,405
20	53.46	20,403	51.19	16,868
25	54.23	25,884	48.19	21,970
30	<u>53.86</u>	28,521	<u>51.09</u>	25,139

results are reusable, and FlexRound cannot reach the high performance of UniQuan_F even though we invest more time.

F.3 Sensitivity Analysis on p , G , and T

We report the change in the performance of quantized models according to the variation of UniQuan_F’s hyperparameters G , T , and p to promote reproducibility. We report the average accuracy of 0-shot and 5-shot MMLU benchmarks of the 3-bit quantized Llama-3 8B models.

Remapping period (p). Figure 6(a) shows the change in the quantized models’ accuracy with regard to the change of remapping period p . We use $p \in \{1, 2, 4, 8, 16, 32\}$ where a higher p represents the sparse update of mapping between weights and quantization levels. As a result, we find that the quantized model achieves the highest accuracy when $p \in \{2, 4\}$, outperforming the case of $p = 1$ where we update the mapping in every step. This result indicates that periodic mapping not only improves the efficiency of UniQuan_F but also im-

proves the accuracy of the quantized models. We recommend using $p = \{2, 4\}$ and we use only $p = 2$ in this paper since they exhibit similar accuracies.

Grid search iterations (G). Figure 6(b) shows the change in the quantized models’ accuracy with regard to the change of grid search iterations G . We use $G \in \{0, 10, 20, 30, 40, 50\}$ where a higher G represents the exhaustive search for unified initialization. We also include $G = 0$ which indicates the case that we find the center of BCQ’s quantization levels using the general alternating update in Algorithm 3, unlike the other cases that use grid search. As a result, we find that $G \in \{0, 30, 40\}$ shows the highest accuracy and we recommend using $G = \{0, 30\}$ to efficiently explore two different strategies for searching the BCQ’s center.

Alternating update iterations (T). Figure 6(c) shows the change in the quantized models’ accuracy with regard to the change of alternating update iterations G . We use $T \in \{5, 10, 15, 20\}$ where a higher T represents the more iterations for alternating update. As a result, we find that UniQuan_F achieves high accuracy when T is equal to or higher than 10. Thus, we recommend using $G = \{10, 15\}$ which shows the highest accuracy.

F.4 Sensitivity on the Clipping Strategies

We compare the performance of three clipping strategies in unified initialization. Table 13 summarizes the comparison of the performance of Llama-3 8B models quantized using the three clipping strategies.

Fixed-maximum and Fixed-minimum strategies outperform the Balanced strategy. This is because the center of BCQ’s quantization level explores more diverse values in Fixed minimum and Fixed maximum strategies than Balanced strategy, result-

Table 13: The average accuracies on 0-shot and 5-shot MMLU benchmarks of 3-bit Llama-3 8B models with various clipping strategies.

Strategy	Accuracy
Fixed-minimum	53.46
Fixed-maximum	54.34
Balanced	53.21

Table 14: The average accuracies on 0-shot and 5-shot MMLU benchmarks of 3-bit Llama-3 8B models on various sizes of sample datasets.

Sample Size	32	64	128	256
Accuracy	47.72	50.71	53.46	53.86

ing in better quantization levels. Therefore, we recommend evaluating Fixed minimum and Fixed maximum strategies and using the better one according to the experimental settings.

F.5 Sensitivity on the Size of Sample Dataset

To illustrate the effect of sample dataset size on the performance of UniQuan_F, we quantize the Llama-3 8B model to 3 bits using sample datasets of varying sizes. We evaluate the performance of quantized models on 0-shot and 5-shot MMLU benchmarks. We use sample datasets ranging in size from 32 to 256, where 128 is the size used in the main text. We summarize the result in Table 14.

Experimental results show that the accuracy of the quantized models increases as the sample size grows. This is attributed to the fact a large number of data points promote the distillation of the general knowledge in the unquantized model to the quantized model. Therefore, UniQuan_F achieves higher performance than that reported in the main text by providing enough data points.

F.6 Flexible Mappings in UniQuan_F

UniQuan_F unifies FlexRound (Lee et al., 2023) and ALTERNATING. We perform an in-depth analysis to verify that UniQuan_F effectively leverages the main optimization technique of FlexRound. The main idea of FlexRound is “flexible mapping” which introduces additional scale factors s and s_r to make weights explore diverse quantization levels, and selects the best one. We compare the amount of flexible mapping occurred in UniQuan_F and FlexRound for validation. We use a 3-bit quantized Llama-3 8B model for analysis.

Case study. We analyze the amount of flexible

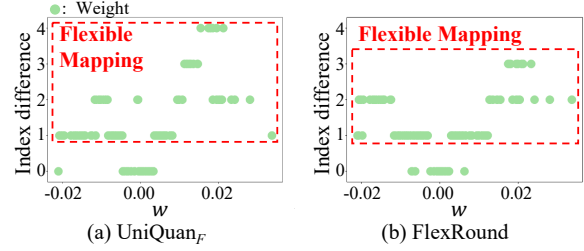


Figure 7: Index difference of mapped quantization levels per weight before and after applying s and s_r . Flexible mappings occur sufficiently in UniQuan_F compared to FlexRound.

Table 15: Proportion of weights exhibiting flexible mapping across the entire model. Each column indicates the index difference of the mapped quantization level resulting from flexible mapping.

Method	Index Difference			
	0	1	2	>2
FlexRound	95.4132	4.5862	5.00e-04	1.30e-05
UniQuan _F	96.0395	3.8619	9.85e-02	7.62e-05

mappings in a weight group of quantized models and visualize the results. Figure 7 represents the index difference of mapped quantization levels for each weight before and after applying scale factors s and s_r when quantizing Llama-3 8B models into 3-bit using UniQuan_F and FlexRound. An index difference equal to or larger than 1 represents that the weight experiences the flexible mapping through s and s_r . Note that plenty of flexible mappings occur in the quantization process of UniQuan_F similarly in FlexRound.

Global pattern. We analyze the proportion of flexible mapping applied across all model weights to examine that flexible mappings occur globally. Table 15 illustrates the proportion of the flexibly mapped weights in the entire model. The column names in the tables indicate the amount of changes in indices of weights’ mapped quantization levels resulting from flexible mapping. Across the entire model, UniQuan_F demonstrates a similar level of flexible mapping as FlexRound. Therefore, UniQuan_F effectively utilizes flexible mapping across the entire model as we intended.

G Use of AI Assistant

We use ChatGPT⁴ and Gemini⁵ only for grammar checking and sentence re-wording purposes. We do

⁴<https://chatgpt.com/>

⁵<https://gemini.google.com/>

not use them for research purposes such as developing our main ideas or analyzing our experimental results.

H Potential Risks

In this paper, we propose UniQuan_F, a quantization method for large language models (LLMs), and there is a potential risk of losing the models' knowledge during quantization. In experiments, we rigorously validate the amount of knowledge loss in both general and task-specific aspects to verify whether the risk occurs. As a result, we demonstrate that UniQuan_F results in significantly less knowledge loss than other methods, confirming its low risk.